Noise Schedule Ablation in Diffusion Models

Group 25 · Foundation Models and Generative AI

Group 25	Name	Roll No.
Member 1	Sameer Sarkar	M26AI1107
Member 2	Joydeep Roy	M26AI1053
Member 3	Krishnendu Halder	M26AI1054
Member 4	Samanwita Patra	M26AI1106

This document has two parts. Part A is our Phase 1 declaration, updated because we had to change the pretrained model (please see the note under item 2). Part B is the Phase 2 submission: it lists what we are submitting and then gives the report, with the sections in the order the assignment asks for. Underwater image enhancement with DiffWater, fine-tuned under the original noise schedule (Run A) and under the power-transformed schedule with x = 2.00 (Run B), compared with the pretrained model (Run 0).

Part A. Phase 1 declaration (revised for DiffWater)

1. Image-to-image task

Paired underwater image enhancement. Given a degraded underwater RGB image (colour cast, haze, low contrast), generate the enhanced reference image of the same scene. The mapping is learned by a conditional diffusion model from paired data. Inputs and outputs are 256 × 256 pixels, which is DiffWater's native size; the EUVP images are resized to this size.

2. Pretrained model

Model	DiffWater: Underwater Image Enhancement Based on Conditional Denoising Diffusion Probabilistic Model (Guan et al., IEEE J. Sel. Topics Appl. Earth Obs. Remote Sens., vol. 17, pp. 2319–2335, 2024)
Paper	https://ieeexplore.ieee.org/document/10365517
Codebase	https://github.com/Guan-MS/DiffWater
Checkpoint	`I390000_E78_gen.pth`, from the `LSUI` folder of the authors' pretrained-weights folder on Google Drive
Data named in the README	LSUI, UIEB, SQUID, U45

Important note: deviation from the declaration. Our Phase 1 declaration named **UW-DiffPhys** (arXiv 2409.18476). We could not use it. Its pretrained checkpoint is shared through a Google Drive link in the authors' repository that we were not able to open, so we had no pretrained weights. We then tried to train the model from scratch, but on a free Colab T4 the runs kept using up the GPU allowance, usually at around the 0th or the 12th epoch. We also expected that training it properly on a T4 would take months, and we looked at renting A100 GPUs but the cost was too high for a course assignment. We wrote to the author on GitHub and on LinkedIn, explained that this is an academic project and asked for access to the checkpoint, but we did not get a reply. So we changed the model, and this time we chose one whose training and inference both fit on a T4. DiffWater is the same kind of model (a conditional DDPM for paired underwater enhancement with $\theta = \bar{\alpha}$) and its weights are public. The task, the dataset, θ and x = 2.00 are unchanged. Only two things differ from the declaration: the model, and the image size, which is 256 × 256 (DiffWater's native size) instead of the declared 128 × 128.

3. Dataset

Dataset	EUVP (Enhancing Underwater Visual Perception), paired subsets; Islam, Xia and Sattar
Source (Kaggle copy used)	https://www.kaggle.com/datasets/pamuduranasinghe/euvp-dataset
Original release	https://irvlab.cs.umn.edu/resources/euvp-dataset

Dataset paper	https://arxiv.org/abs/1903.09766
Subsets used	Underwater Dark (5,550 training pairs) and Underwater Scenes (2,185 training pairs); all 7,735 pairs were matched by file name. The Underwater ImageNet subset and the EUVP *validation* folders were not used.

4. Evidence that the dataset was not used in pretraining

Source	Where	What it says
Paper	Section IV-A “Implementation Details”, p. 2327, right column, lines 1–2 (the sentence starts on the last line of the left column)	Training used 5004 paired images from LSUI and 800 paired images from UIEB.
Paper	Section IV-B “Datasets and Evaluation Metrics”, p. 2327 right column lines 1–3 of the section; p. 2328 left column lines 1–22 (UIEB, LSUI, SQUID) and right column lines 1–6 (U45)	The experiments use four public datasets: UIEB, LSUI, SQUID and U45. UIEB gives 800 training pairs and the test sets TEST-U90 and TEST-C60; LSUI (5004 pairs) was used for training; SQUID and U45 are test sets.
Paper	Whole paper (text search of all 17 pages)	The word EUVP does not appear anywhere.
Codebase	README.md, section “Dataset”	Lists LSUI, UIEB, SQUID and U45 as the data.

Argument for a different source. The DiffWater paper describes its data as real underwater images from UIEB and LSUI with reference images, plus the SQUID and U45 test sets. EUVP comes from a different collection process. Its authors took images with seven cameras (several GoPros, uEye cameras on an AUV, low-light USB cameras and an ROV camera) during ocean trips and robot experiments, and added frames from a few YouTube videos (EUVP paper, Section 3.3). For the paired part, they trained a CycleGAN-based model and used it to distort good-quality images so that each good image gets a degraded partner (same section). So the pairs in EUVP are built differently from UIEB and LSUI. The paper compares DiffWater with FUnIE-GAN (reference [32], by Islam et al., the group that made EUVP), but only as one of the methods in the comparison; it does not use the EUVP data.

5. Native noise parameterization

$\boldsymbol{\theta_t = \bar{\alpha}_t}$, the cumulative product of the per-step retention coefficients $\alpha_t = 1 - \beta_t$.

The paper defines $\bar{\alpha}_t := \prod_{s=1..t} \alpha_s$ in Section III-C “DiffWater: Diffusion Process” and writes it as $\gamma_t = \bar{\alpha}_t$.

The code is in `model/ddpm_modules/diffusion.py`, lines 114–116.

The forward process is Eqs. (4)–(8) of the paper:

$$\mathbf{q(x_t \mid x_{t-1}) = N(\sqrt{(1-\beta_t)}\, x_{t-1},\ \beta_t\, I)} \text{ (Eq. (5))},$$

with the closed form $\mathbf{x_t = \sqrt{\gamma_t}\, x_0 + \sqrt{(1-\gamma_t)}\, \varepsilon}$ (Eq. (8)), for t = 1, …, T with T = 2000.

Training (Algorithm 1 and Eq. (17)) and ancestral sampling (Eqs. (12)–(16)) use $\gamma_t = \bar{\alpha}_t$ directly.

Confirmation that $\boldsymbol{\theta_t \in (0, 1)}$ for all t.

The paper states $\boldsymbol{\beta_t \in (0, 1)}$ and uses a linear schedule from $\mathbf{10^{-6}}$ **to** $\mathbf{10^{-2}}$ **with T = 2000**; the code uses the same values (`config/config.yml`, lines 44–54). So every $\alpha_t = 1 - \beta_t$ lies between 0.99 and 0.999999, inside (0, 1).

A finite product of numbers in (0, 1) is in (0, 1), so $\bar{\alpha}_t \in (0, 1)$ for every t, and it is strictly decreasing because $\bar{\alpha}_t = \bar{\alpha}_{t-1}\, \alpha_t < \bar{\alpha}_{t-1}$. The run logs print $\bar{\alpha}_T = 4.386 \times 10^{-5}$.

Modified schedule:

$\{\theta_t^x\} = \{\bar{\alpha}_t^x\}$ is also in (0, 1) and strictly decreasing, and its per-step form $\alpha'_t = \bar{\alpha}_t^x / \bar{\alpha}_{t-1}^x = \alpha_t^x$ is in (0, 1) as well, so no conversion to another parameterization is needed. Computed: $\bar{\alpha}_T^2 = 1.924 \times 10^{-9}$, and β'_t runs from 2.0×10^{-6} to 0.0199.

6. Chosen exponent

x = 2.00, inside the allowed interval (1.5, 3.0). Run B uses the schedule $\{\bar{\alpha}_t^{2.00}\}$, applied elementwise.

Part B. Phase 2 submission

Submission components

1. **Code.** The code is in the zip file submitted together with this report. `README.md` inside the zip gives the exact commands for Run 0, Run A and Run B, the random seed and the software environment. Its section "What is in this folder" lists the new code: `finetune.py` (training, sampling, logging and the schedule change, with the exponent `X = 2.00` as a named constant at the top), `val_loss.py` (the post-hoc validation), and the Colab notebooks we used to run everything. Changed lines are marked `# MODIFIED` and added code is marked `# ADDED` (details in Section 5).
2. **Fine-tuning logs.** One CSV per fine-tuning run with the columns `epoch` and `train_loss`, and the other logs, are in the zip: `logs/RunA/train_log.csv` and `logs/RunB/train_log.csv`. The samples at five equally spaced epochs (2, 4, 6, 8, 10; three test images each) are in `samples/RunA/` and `samples/RunB/`, in one folder per epoch. Run 0 files are in `logs/Run0/ and samples/Run0/`.
3. **Inference trajectories.** For the final models of Run A and Run B and for test images 1 to 3, the noisy sample x_t at five timesteps (t = 1999, 1499, 1000, 500 and the final x_0) is saved as one strip per image, `idxNN_traj.png`, in the `epoch_10` folders above. They are shown in Figure 5. Each timestep is also saved as a separate file, labelled with run and timestep, in trajectories/.

Report

1. Task and Model

Task. Paired underwater image enhancement. A degraded underwater image (colour cast, haze, low contrast) is the input and the enhanced image that EUVP provides for the same scene is the target. A conditional diffusion model learns this mapping from pairs of images. All images are 256 × 256.

Model. DiffWater (Guan et al., 2024; paper https://ieeexplore.ieee.org/document/10365517; code https://github.com/Guan-MS/DiffWater, Apache-2.0) is a conditional denoising diffusion probabilistic model (DDPM) for underwater image enhancement. The denoiser is the SR3-style U-Net (155.3 million parameters, counted by us). Its input has 6 channels: the 3-channel condition y, which is the degraded image after the paper's colour channel compensation (3C), and the 3-channel noisy target x_t. The network also gets the integer timestep t through a sinusoidal embedding, and it is trained to predict the noise ε with an L1 loss. At test time it starts from pure noise and removes it step by step for all T = 2000 steps (ancestral sampling). We used the pretrained checkpoint `I390000_E78_gen.pth` from the `LSUI` folder of the authors' pretrained-weights folder.

Forward process and the native parameter θ_t. The forward process adds a little Gaussian noise at each of the T = 2000 steps. The amount added at step t is β_t, which grows linearly from 10^{-6} to 10^{-2}. Written with the symbols of the paper (equation numbers in brackets are the paper's):

$$q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) = \mathcal{N}\left(\mathbf{x}_t;\ \sqrt{1-\beta_t}\ \mathbf{x}_{t-1},\ \beta_t \mathbf{I}\right) \tag{1}$$

$$\alpha_t = 1 - \beta_t, \qquad \bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s \tag{2}$$

$$q(\mathbf{x}_t \mid \mathbf{x}_0) = \mathcal{N}\left(\mathbf{x}_t;\ \sqrt{\bar{\alpha}_t}\ \mathbf{x}_0,\ (1-\bar{\alpha}_t)\mathbf{I}\right) \tag{3}$$

$$\mathbf{x}_t = \sqrt{\gamma_t}\ \mathbf{x}_0 + \sqrt{1-\gamma_t}\ \boldsymbol{\varepsilon}, \qquad \gamma_t = \bar{\alpha}_t, \quad \boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \tag{4}$$

Here x_0 is the clean target image, x_t is the noisy image at step t, I is the identity matrix, ε is standard Gaussian noise, β_t is the noise variance added at step t, $\alpha_t = 1 - \beta_t$ is the fraction of the signal kept at step t, and $\bar{\alpha}_t$ is the product of the first t of these fractions, so it tells how much of the original signal is left at step t. The paper renames $\bar{\alpha}_t$ as γ_t to keep the notation short. Eq. (1) is Eq. (5) of the paper, Eq. (2) is the text above Eq. (6) and Eq. (3) is Eq. (6); Eq. (4) is Eqs. (7) and (8).

Training and sampling use the same quantity. The network f_θ (here θ in f_θ means the network weights, not our noise parameter) is trained with the L1 noise-prediction loss of Eq. (17) of the paper, with p = 1, and one reverse step is Eq. (16) of the paper (Algorithm 2, line 5). In both, $\bar{\alpha}_t$ enters through γ_t, and α_t through the ratio $\bar{\alpha}_t / \bar{\alpha}_{t-1}$:

$$\text{Loss} = \mathbb{E}_{(\mathbf{x},\mathbf{y}),\,\boldsymbol{\varepsilon},\,t} \left\| f_\theta\left(\mathbf{y},\ \sqrt{\gamma_t}\,\mathbf{x}_0 + \sqrt{1-\gamma_t}\,\boldsymbol{\varepsilon},\ t\right) - \boldsymbol{\varepsilon} \right\|_1 \tag{5}$$

$$\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\gamma_t}} f_\theta(\mathbf{y}, \mathbf{x}_t, t)\right) + \sqrt{\frac{1-\gamma_{t-1}}{1-\gamma_t}\beta_t}\ \mathbf{z} \tag{6}$$

In Eq. (5), y is the colour-compensated condition image and t is drawn uniformly from 1, …, T; in Eq. (6), z is fresh Gaussian noise for t > 1 and z = 0 for t = 1. **So $\theta_t = \bar{\alpha}_t = \gamma_t$ is the native noise parameter.** Run A uses θ_t. Run B uses $\theta_t^x = \bar{\alpha}_t^x$ with x = 2.00.

2. Dataset and Verification

Dataset. EUVP, *Enhancing Underwater Visual Perception* (Islam, Xia and Sattar; paper https://arxiv.org/abs/1903.09766; release https://irvlab.cs.umn.edu/resources/euvp-dataset; we used the Kaggle copy https://www.kaggle.com/datasets/pamuduranasinghe/euvp-dataset). We use two of its paired subsets: **Underwater Dark** with 5,550 training pairs and **Underwater Scenes** with 2,185 training pairs, so 7,735 pairs that matched by file name (`trainA` is the degraded image, `trainB` the target). The Underwater ImageNet subset is left out. Our script resizes every image to **256 × 256** (LANCZOS), the size DiffWater works at.

Why it fits the task. Every pair has a degraded underwater image and a better-looking image of the same scene, which is exactly the input and target of our task, and the images show the problems we want to fix (colour cast, haze, low contrast). One thing to keep in mind for the metrics: according to the EUVP paper (Section 3.3), the degraded image of a pair was produced by a CycleGAN-based model from a good-quality image, and the authors say their goal is perceptual enhancement, not modelling the physical degradation. So the target is a good-looking image, not a physically measured ground truth.

What we used. 800 training pairs (575 Dark and 225 Scenes) and 30 held-out test pairs, all Dark. The EUVP *validation* folders cannot be used for testing: they have no reference images (only 1 of the 570 Dark names matches a training reference) and some of their files are byte-identical to training files (10 Dark, 1 Scenes). The exact split procedure is in Section 4.

Evidence that EUVP was not used to pretrain DiffWater. This is the evidence from Part A, item 4, in short. (i) Paper, Section IV-A, p. 2327, right column, lines 1–2: DiffWater was trained on 5004 LSUI pairs and 800 UIEB pairs. (ii) Paper, Section IV-B, p. 2327 right column and p. 2328 (left column lines 1–22, right column lines 1–6): the experiments use four public datasets, UIEB, LSUI, SQUID and U45; EUVP is not mentioned anywhere in the 17 pages. (iii) Codebase, README “Dataset”: LSUI, UIEB, SQUID and U45. (iv) EUVP was collected with the authors' own cameras and YouTube frames, and its paired images were generated with a CycleGAN-based model (EUVP paper, Section 3.3), which is a different source from the real images of UIEB and LSUI. One caveat, about ImageNet-style file names, is discussed in Section 7, so we treat EUVP as *very probably* but not certainly outside the pretraining distribution.

3. Schedule Analysis

$\theta_t \in (0, 1)$ for all t. Every β_t is between 10^{-6} and 10^{-2}, so every $\alpha_t = 1 - \beta_t$ is between 0.99 and 1. The product of finitely many numbers in (0, 1) stays in (0, 1), so $\bar{\alpha}_t \in (0, 1)$, and it decreases at every step because each new factor is below 1. Raising a number in (0, 1) to the power x = 2 keeps it in (0, 1) and keeps the order, so $\bar{\alpha}_t^2$ is also in (0, 1) and strictly decreasing. We computed $\bar{\alpha}_1 = 0.999999$, $\bar{\alpha}_T = 4.386\times10^{-5}$ (Run A) and $\bar{\alpha}_T^2 = 1.924\times10^{-9}$ (Run B). Both are positive and can be stored in float32. The script checks this on every run (`apply_and_check_schedule`). Figure 1(a) plots θ_t and θ_t^x against t on the same axes.

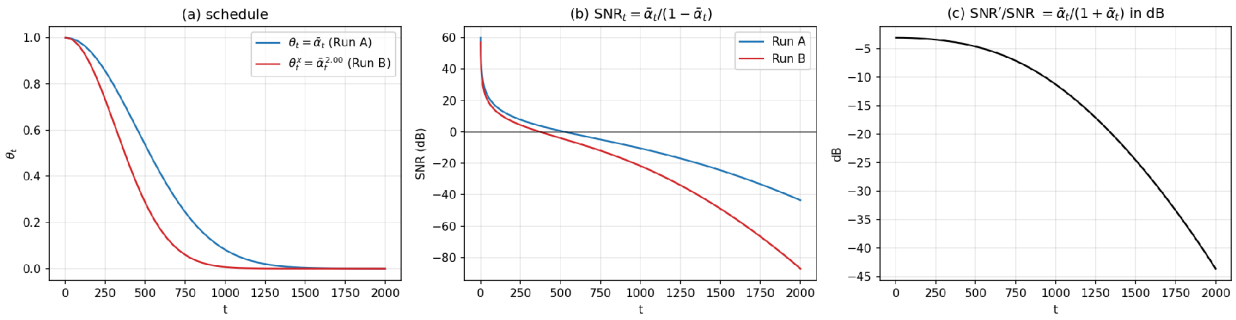


Figure 1: (a) $\theta_t = \bar{\alpha}_t$ and θ_t^x on the same axes; (b) the signal-to-noise ratio of both schedules; (c) the ratio SNR′/SNR.

What changes in the forward process when x > 1. Because $\bar{\alpha}_t < 1$, we have $\bar{\alpha}_t^x < \bar{\alpha}_t$ at every $t > 0$, and the gap grows with x. The noisy sample becomes

$$\mathbf{x}_t = \bar{\alpha}_t^{x/2}\,\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t^{x}}\;\boldsymbol{\varepsilon} \tag{7}$$

so the signal coefficient shrinks faster and the noise variance $1 - \bar{\alpha}_t^x$ is larger at the same t. The per-step coefficients follow from the new product:

$$\alpha_t' = \frac{\bar{\alpha}_t^{x}}{\bar{\alpha}_{t-1}^{x}} = \alpha_t^{x}, \qquad \beta_t' = 1-(1-\beta_t)^x \approx x\beta_t \tag{8}$$

For x = 2, $\beta'_t = 2\beta_t - \beta_t^2$, which runs from 2.0×10^{-6} to 0.0199 (Run A: 10^{-6} to 0.01). In plain words, the image is destroyed in fewer steps. The table gives some numbers (t counted from 1, values from the computed schedule).

	Run A: θ_t	**Run B: θ_t^2**
$\bar{\alpha}$ at t = 250 / 500 / 1000	0.8545 / 0.5339 / 0.0814	0.7301 / 0.2851 / 0.0066
first t where θ (A) or θ^2 (B) is below 0.5, i.e. SNR below 0 dB	527	373
number of steps with $\bar{\alpha} < 0.01$ (almost pure noise)	645 of 2000 (32 %)	1041 of 2000 (52 %)
largest $1/\sqrt{\bar{\alpha}}$ (error amplification, see Section 7)	151	22,800

Table 1: the two schedules in numbers.

Signal-to-noise ratio. The SNR at step t compares how much signal and how much noise there is in x_t. For the original schedule and for the power x it is

$$\mathrm{SNR}_t = \frac{\bar{\alpha}_t}{1-\bar{\alpha}_t}, \qquad \mathrm{SNR}_t' = \frac{\bar{\alpha}_t^{x}}{1-\bar{\alpha}_t^{x}} \tag{9}$$

An SNR of 0 dB means signal and noise are equally strong. For x = 2 this can be simplified:

$$\mathrm{SNR}_t' = \frac{\bar{\alpha}_t^2}{(1-\bar{\alpha}_t)(1+\bar{\alpha}_t)} = \mathrm{SNR}_t \cdot \frac{\bar{\alpha}_t}{1+\bar{\alpha}_t} \qquad (x=2) \tag{10}$$

The factor $\bar{\alpha}_t/(1 + \bar{\alpha}_t)$ is always below 1/2, so at every step the SNR of Run B is at least 3 dB lower than that of Run A. The loss is 3.0 dB at t = 1, 4.6 dB at t = 500, 11.2 dB at t = 1000 and 43.6 dB at t = T (Figure 1(c)). So the same timestep index means a noisier input in Run B than in Run A. This is the main thing that the pretrained network has to cope with in Run B, because it learned what each t means with the original schedule.

4. Experimental Setup

Dataset preparation

1. We downloaded the Kaggle copy of EUVP on Colab. In the Dark and Scenes folders we matched `trainA` (degraded) and `trainB` (target) by file name, and computed for every image an MD5 hash and a 64-bit difference hash (dHash).
2. **Test set.** Shuffle the Dark pairs with seed 1234, take the first 30 that are not byte-identical to each other, and sort them. The list is saved as `test_split_dark30.json` (split fingerprint `f8cc4714a30c`).
3. **Training pool.** All other Dark and Scenes pairs, minus exact duplicates (5 found) and near-duplicates (dHash distance of at most 6 out of 64 bits; 0 found) of any test image. This leaves 7,700 pairs.
4. **Training set.** Shuffle the pool with seed 1234 and take 800 pairs, which gave 575 Dark and 225 Scenes (fingerprint `6a9f33de995c`).
5. Resize all images to 256 × 256 (LANCZOS) and save them as PNG. DiffWater's loader then applies the colour channel compensation to the input; there is no augmentation.

Train, validation and test split

Train 800 pairs, test 30 pairs, and no validation set during fine-tuning. A set of 200 validation pairs was scored afterwards (post hoc, described below). The split is checked in every Colab session with the two fingerprints. We did not remove pairs that show the same scene in a different frame; the EUVP release has the same limitation.

Fine-tuning hyperparameters (identical for Run A and Run B)

Item	Value
Starting weights	`I390000_E78_gen.pth`, the same file for both runs
Optimizer	Adam (fresh state), learning rate 1e-5 (the authors pretrained with 3e-6 and batch size 1)
Batch size	4 (800 pairs, so 200 steps per epoch)
Epochs	10 (2000 optimisation steps)
Loss	L1 between the predicted and the true noise, as in the repository
EMA, augmentation	none (the repository's training path has neither)
Seed	1234. Epoch e is seeded with 1234 + e, which fixes the data order, the timesteps and the noise. Test image k is sampled with seed 1234 × 1000 + k.
Sampler	ancestral, all 2000 steps, same code and seeds in Run 0, A and B
Exponent	Run 0 and Run A: 1. Run B: `X = 2.00` (named constant at the top of `finetune.py`)

Because the seeds are the same, Run A and Run B see the same batches, the same timesteps and the same noise. Only the schedule is different. A and B also used the same 30 test images, in the same order, as Run 0.

How validation was performed during fine-tuning

There was no separate validation set. During fine-tuning we looked at the training loss (every epoch, saved in the CSV of each run) and we generated samples for the first 3 of the 30 test images at epochs 2, 4, 6, 8 and 10 and computed PSNR and SSIM for them. These samples were only looked at. No checkpoint, epoch or hyperparameter was chosen from them, and the model that we evaluate is the epoch-10 model in both runs. It does mean that these 3 images are part of the test set.

Post-hoc validation (added after the runs; this is not validation during fine-tuning). We kept no per-epoch weights, so validation during fine-tuning cannot be redone. After Run A and Run B had finished we scored the pretrained weights and each run's final weights on **200 held-out validation pairs** (137 Dark and 63 Scenes). They are the pairs that follow the 800 training pairs in the seeded shuffle of the pool, minus exact and near-duplicates of any training image (0 exact and 1 near-duplicate were skipped). We checked that they do not overlap with the training or the test pairs, and both Colab accounts built the

same set (fingerprint `24416ebc2732`). The score is the training objective itself, the mean absolute error between the true noise and the noise the network predicts, under the schedule of that run. Each image is scored at 10 fixed timesteps (t = 100, 300, …, 1900) with fixed noise that is identical for the pretrained and the fine-tuned weights, so the comparison is paired per image:

$$L_{\text{val}} = \frac{1}{NK} \sum_{i=1}^{N} \sum_{k=1}^{K} \text{mean} \left| \boldsymbol{\varepsilon}_{ik} - f_\theta(\mathbf{y}_i, \mathbf{x}_{t_k}, t_k) \right| \tag{11}$$

Here N = 200 is the number of images, K = 10 the number of timesteps, y_i the condition image, ε_{ik} the fixed noise and x_{tk} the noisy target at timestep t_k. Because the grid of 10 timesteps is coarse and the loss falls steeply with t, the absolute value of L_{val} is not the same as the training loss; only the differences between pretrained and fine-tuned weights are meaningful. Nothing in Run A or Run B was chosen from this score. The code is `val_loss.py`.

How the test set was used

After training, all 30 test images were sampled with the epoch-10 model of each run (Run 0: with the pretrained weights) and scored against their references. Sampling one image takes about 5 minutes on a T4, because it needs 2000 network evaluations.

Metric

We score every generated image against its reference with two full-reference measures, PSNR and SSIM from scikit-image (`data_range=255`, `channel_axis=2`), on 256 × 256 images with 8-bit RGB values.

PSNR (peak signal-to-noise ratio) compares two images pixel by pixel. It takes the mean squared error (MSE) between the generated image and the reference and converts it to decibels:

$$\text{PSNR} = 10 \log_{10} \frac{255^2}{\text{MSE}} \tag{12}$$

A higher PSNR means that the pixel values are closer to the reference. Because it looks at every pixel, a colour cast that shifts many pixels in the same direction lowers it a lot.

SSIM (structural similarity) compares small windows of the two images a and b by their mean brightness (μ), their contrast (σ^2) and how they vary together (σ_{ab}); c_1 and c_2 are small constants that avoid division by zero:

$$\text{SSIM}(a, b) = \frac{(2\mu_a\mu_b + c_1)(2\sigma_{ab} + c_2)}{(\mu_a^2 + \mu_b^2 + c_1)(\sigma_a^2 + \sigma_b^2 + c_2)} \tag{13}$$

SSIM can take negative values; 1 indicates identical images. It reacts less to a uniform colour shift than PSNR does, because it is more about structure. **We use PSNR and SSIM because they are the two full-reference metrics that the DiffWater paper itself uses on paired data (Section IV-B, p. 2328) and our test pairs come with reference images.**

Environment

Google Colab, Tesla T4 GPU, Python 3.13.16, PyTorch 2.11.0+cu130, DiffWater commit `7db68f34060bbf1d7c5b8153664fb16a3dcda1ae`, `finetune.py` md5 `38c97203e8ccafdfb80cb2be74b0bb3a` (checked at the start of every session).

- Run 0 took about 162 minutes,
- Run B about 270 minutes
- Run A about 280 minutes of GPU time, on a T4.

How the runs were executed

Run 0 and Run B each ran in one Colab session. Run A needed four sessions, because Colab disconnected us and some accounts ran out of GPU quota. In the first session Run A trained all 10 epochs, but its last resume file never reached Drive, so the second session resumed from the epoch-8 file

and trained epochs 9 and 10 again (the epoch-9 loss was 0.06070 both times, the epoch-10 loss 0.06372 and 0.06355). Test images 25 to 30 were sampled in two more sessions in other Google accounts, which resumed from the same epoch-10 file through a shared Drive folder. The weights, seeds, script, commit and data fingerprints were the same in every session, and the launch cell checks them before it starts.

5. Implementation

Files. We wrote one new file, `finetune.py`. It has the training loop, the sampler wrapper, the logging and the schedule change. No file of DiffWater was edited on disk. At run time `finetune.py` replaces one method, `GaussianDiffusion.set_new_noise_schedule` in `model/ddpm_modules/diffusion.py`, by a copy in which the five lines 86, 87, 89, 90 and 91 are marked `# MODIFIED`; the header comment of the copy (line 73) and the line that installs it (line 289) are marked `# MODIFIED` too. The rest of the file is new code, and each new block starts with an `# ADDED` comment; the file header says that everything outside the copied method is added. The exponent is the named constant `X = 2.00` at line 46. The post-hoc validation is in a second new file, `val_loss.py`, which imports `finetune.py` unchanged.

Original and modified lines side by side.

Original (DiffWater, `diffusion.py`, lines 114–116)	Modified (`finetune.py`, lines 84–91)

`alphas = 1. - betas`
`alphas_cumprod = np.cumprod(alphas, axis=0)`
`alphas_cumprod_prev = np.append(1., alphas_cumprod[:-1])`
	`alphas = 1. - betas`
	`alphas_cumprod = np.cumprod(alphas, axis=0)`
	`if EXPONENT != 1.0:  # MODIFIED`
	`  alphas_cumprod = alphas_cumprod ** EXPONENT  # MODIFIED`
	`alphas_cumprod_prev = np.append(1., alphas_cumprod[:-1])`
	`if EXPONENT != 1.0:  # MODIFIED`
	`  alphas = alphas_cumprod / alphas_cumprod_prev  # MODIFIED`
	`  betas = 1. - alphas  # MODIFIED`

Table 2: the schedule lines of DiffWater and our modified copy (explanatory comments of the real file left out).

All other schedule arrays (the square roots, the posterior variance and the posterior mean coefficients) are computed from `alphas_cumprod` further down in the same method, unchanged, so noising during training and sampling both use the new schedule. When the exponent is 1 the new code is not used, so Run 0 and Run A run the original path.

How we verified that Run B used θ_t^x and not θ_t.

1. **A trap we found and fixed.** The schedule arrays are registered buffers, so they are stored inside the checkpoint's `state_dict`, and loading the pretrained checkpoint silently overwrites a modified schedule with the original one. A small demonstration in our notebook shows two buffer values 0.81 and 0.25 turning back into 0.90 and 0.50 after `load_state_dict`. `finetune.py` therefore sets the schedule again after every load.
2. **An assertion after every load** (`apply_and_check_schedule`): the buffers must equal $\bar{\alpha}_t^{\text{EXPONENT}}$ within a relative tolerance of 1e-5, lie in (0, 1) and decrease, and the per-step β must lie in (0, 1). The run stops otherwise.
3. **The log prints the schedule.** The first lines of Run B read `exponent 2.0, abar_last 1.924e-09, beta_min 2.0e-06, beta_max 0.0199`; those of Run A read `exponent 1.0, abar_last 4.386e-05, beta_min 1.0e-06, beta_max 0.01`. These are the values of Section 3, and `run_config.json` stores them.
4. **The pictures agree.** At the same t, the trajectory images of Run B are noisier than those of Run A (Figure 5).
5. For the post-hoc validation, `val_loss.py` also reads the schedule that is stored inside each `final_gen.pth` and stops if it is not the one of that run (it printed `abar_last 4.3860e-05` for A and `1.9237e-09` for B).

6. Results

Run 0: output samples of the pretrained model on our input images

Figure 2 shows the output of Run 0 on test images 1 to 3, next to the input and the reference. These three images are in `samples/Run0/` in the zip, and the scores of all 30 test images are in `logs/Run0/eval_metrics.csv`.

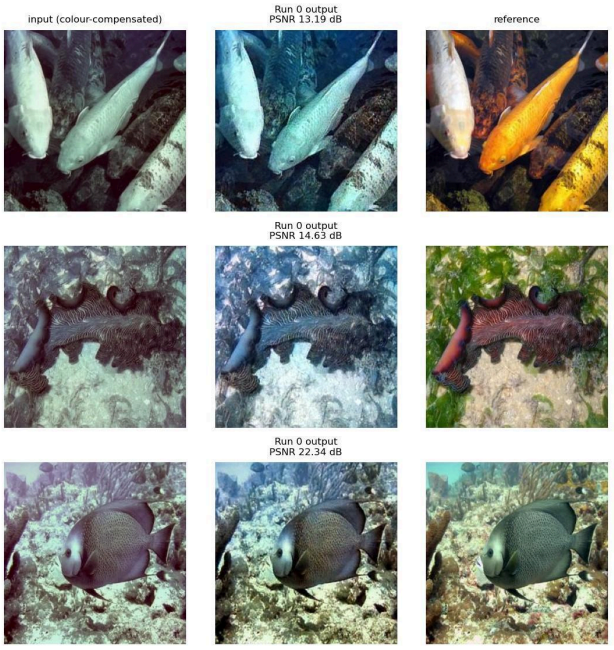


Figure 2: Run 0 (pretrained DiffWater, no fine-tuning) on test images 1 to 3: the input (after colour compensation), the output of Run 0 and the reference. The PSNR of each output is printed above it.

The input shown is the colour-compensated input, because the loader applies the compensation before the network sees the image.

Fine-tuning loss of Run A and Run B

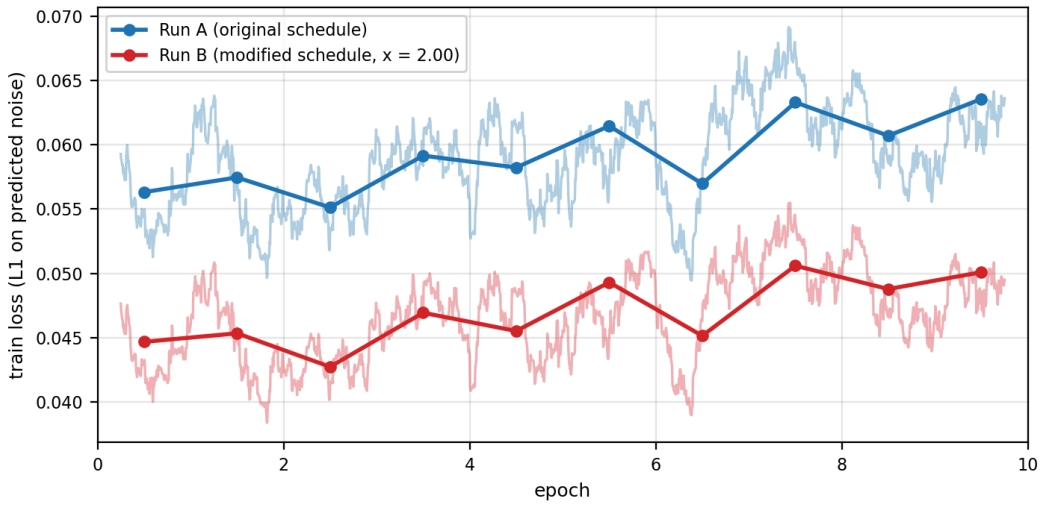


Figure 3: fine-tuning loss of Run A and Run B on one plot. Thin lines are 100-step moving means of the step losses, dots are the per-epoch means that are in the two CSV files (`train_log.csv`).

Epoch	Run A train_loss	Run B train_loss
1	0.0563	0.0447
2	0.0575	0.0453
3	0.0551	0.0427
4	0.0591	0.0469
5	0.0582	0.0455
6	0.0615	0.0493
7	0.0570	0.0451
8	0.0633	0.0506
9	0.0607	0.0488

Epoch	Run A train_loss	Run B train_loss
10	0.0636	0.0501

Table 3: the per-epoch training loss (the content of the two CSV files).

The losses of the two runs are not comparable in level, because the same t has a different noise level in each schedule. Both runs move up and down together, which is expected, since they use the same batches, timesteps and noise in each epoch.

Generated samples at the five checkpoints

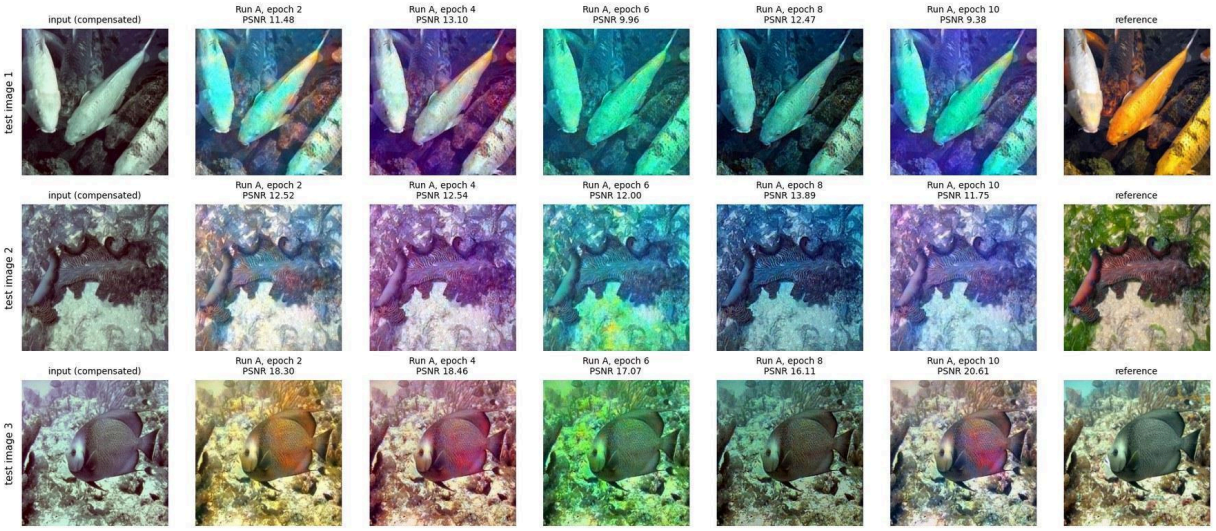


Figure 4a: Run A, generated samples for test images 1 to 3 at epochs 2, 4, 6, 8 and 10 (columns), with input and reference.

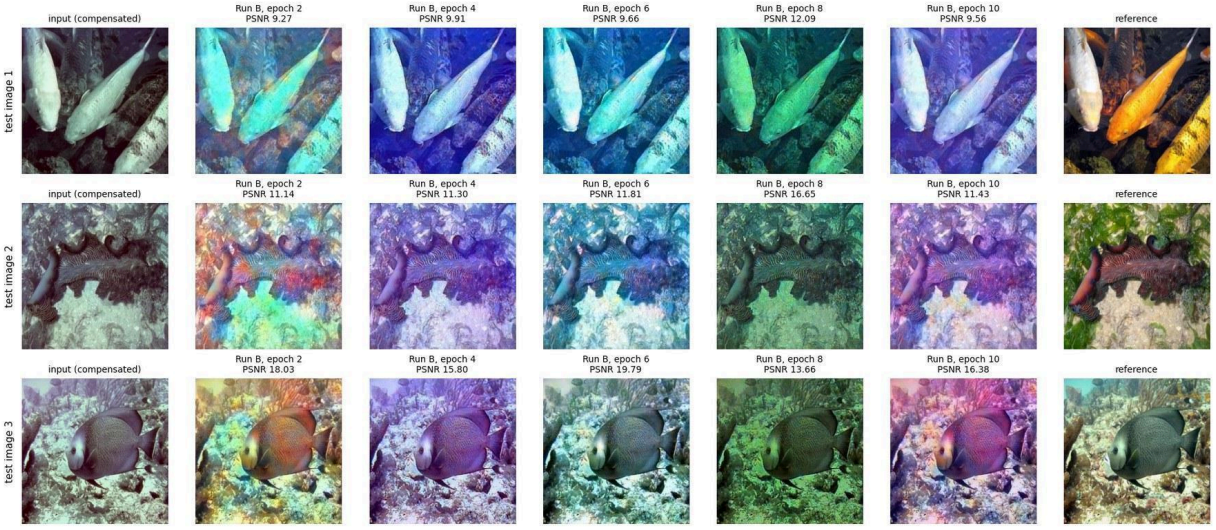


Figure 4b: Run B, generated samples for test images 1 to 3 at epochs 2, 4, 6, 8 and 10 (columns), with input and reference.

Epoch	Run A: PSNR / SSIM	Run B: PSNR / SSIM
2	14.10 / 0.780	12.81 / 0.701
4	14.70 / 0.800	12.34 / 0.771
6	13.01 / 0.706	13.75 / 0.759
8	14.16 / 0.762	14.13 / 0.742
10	13.91 / 0.771	12.46 / 0.774

Table 4: mean PSNR (dB) and SSIM over the same 3 test images at each checkpoint (output against reference).

Inference trajectories of the final models

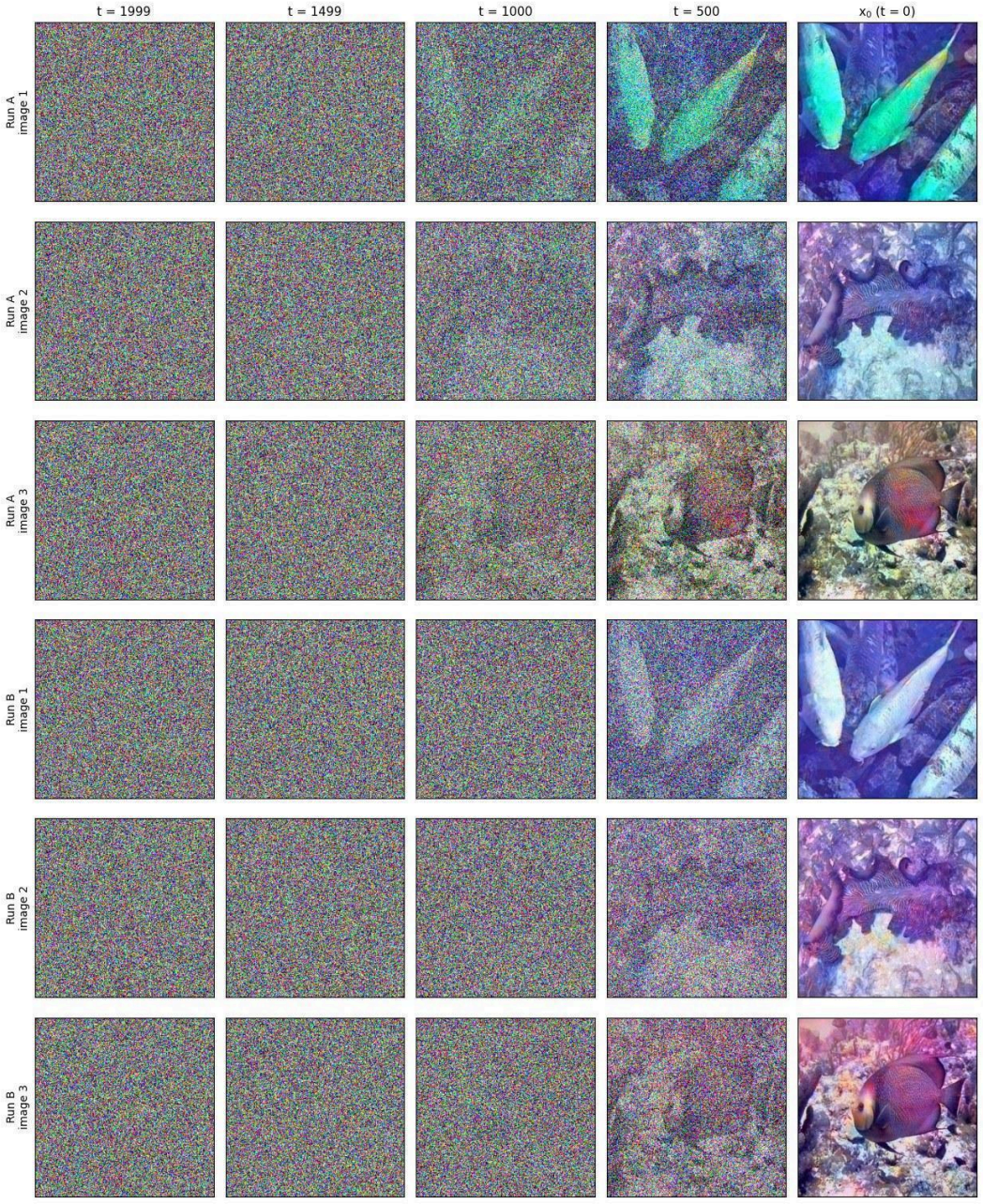


Figure 5: inference trajectories of the final models of Run A and Run B: the noisy sample x_t at t = 1999, 1499, 1000 and 500 and the final x_0 (right column), for test images 1 to 3.

Test-set scores

The table gives the mean and the standard deviation over the 30 held-out test pairs. We use PSNR and SSIM because they are the two full-reference metrics that the DiffWater paper itself uses on paired data, and our test pairs come with references.

Condition	PSNR (dB), mean ± sd	SSIM, mean ± sd
Input (colour-compensated), no model	20.65 ± 4.13	0.889 ± 0.081
Run 0 (pretrained, no fine-tuning)	16.91 ± 3.07	0.792 ± 0.071
Run A (fine-tuned, θ_t)	14.36 ± 3.02	0.770 ± 0.112
Run B (fine-tuned, θ_t^2, x = 2.00)	12.85 ± 2.19	0.763 ± 0.097

Table 5: test-set scores on the 30 held-out Underwater Dark pairs, output against reference.

Comparison (first minus second)	PSNR difference (s.e.)	SSIM difference (s.e.)
Run A − Run 0	−2.55 dB (0.53), distinguishable; first run higher on 6/30	−0.022 (0.014), not distinguishable
Run B − Run 0	−4.06 dB (0.54), distinguishable; first run higher on 3/30	−0.029 (0.012), borderline (2.4 s.e.)
Run B − Run A	−1.51 dB (0.40), distinguishable; first run higher on 5/30	−0.007 (0.008), not distinguishable

Table 6: paired differences over the same 30 images. A mean difference below 2 standard errors (s.e.) counts as not distinguishable from zero.

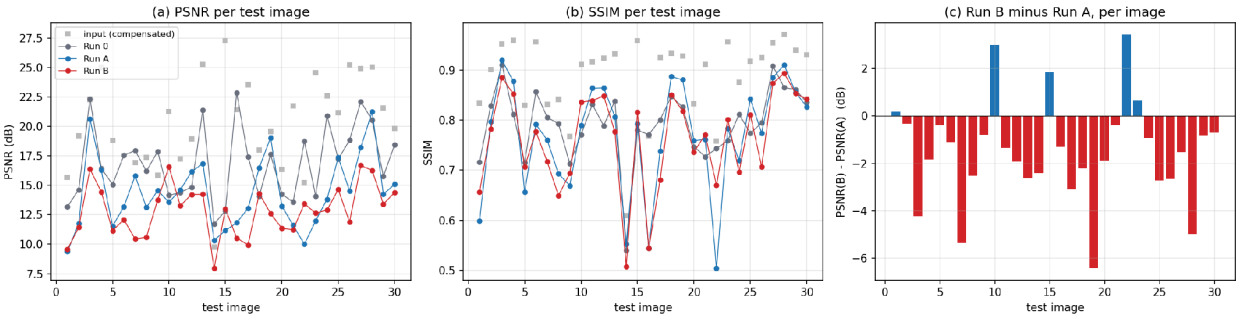


Figure 6: (a) PSNR and (b) SSIM per test image for the colour-compensated input, Run 0, Run A and Run B; (c) PSNR of Run B minus PSNR of Run A, per image.

Post-hoc validation: noise-prediction loss on 200 held-out pairs

This was run after fine-tuning and nothing was chosen from it (Section 4). Each run is compared with the pretrained weights under the same schedule. The Run A and Run B columns cannot be compared with each other, because the same timestep is a different noise level in the two schedules.

	Run A (θ_t)	Run B (θ_t^2, x = 2.00)
pretrained weights	0.05559	0.07859
fine-tuned weights (epoch 10)	0.05253	0.03922
difference, fine-tuned minus pretrained (s.e.)	−0.00306 (0.00039), about −5.5 %	−0.03937 (0.00034), about −50.1 %
images on which fine-tuned is lower	130 of 200	200 of 200
timesteps (of 10) at which fine-tuned is lower	6 (t = 100 to 1100); higher at t = 1500, 1700 and 1900	all 10

Table 7: post-hoc validation loss (mean absolute error of the predicted noise, mean over 10 timesteps and 200 pairs).

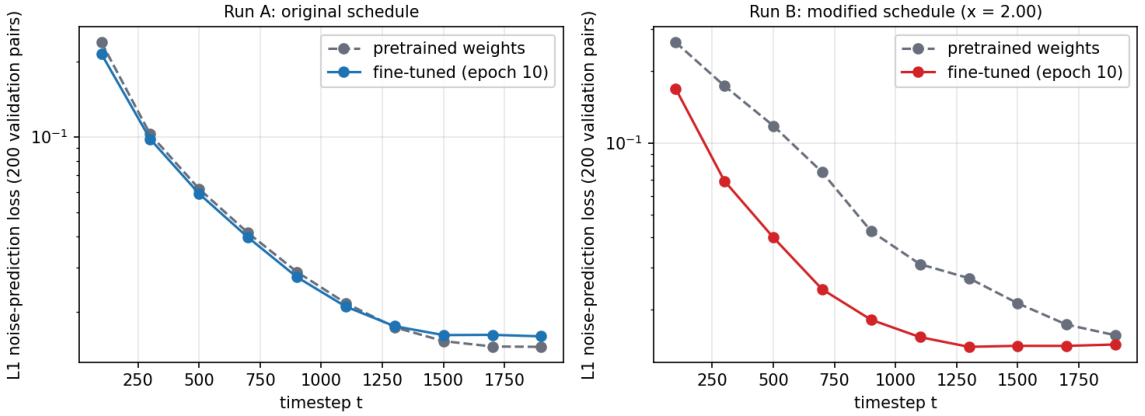


Figure 7: validation loss per timestep for the pretrained and the fine-tuned weights, Run A (left) and Run B (right), on a log scale.

So fine-tuning improved the training objective on unseen pairs under both schedules, and much more under the modified schedule, where the pretrained network starts further from the right answer. In Run A

the improvement is only at small and middle noise levels; near pure noise (t ≥ 1500) the fine-tuned model is a little worse, by 6 % to 11 %. For Run A the score of the random-timestep forward pass used in training was also lower for the fine-tuned weights in all three seeds we tried, and its mean on the validation images (0.0646) is close to the epoch-10 training loss (0.0636), so we see no sign of a gap between training and validation pairs.

The three conditions side by side

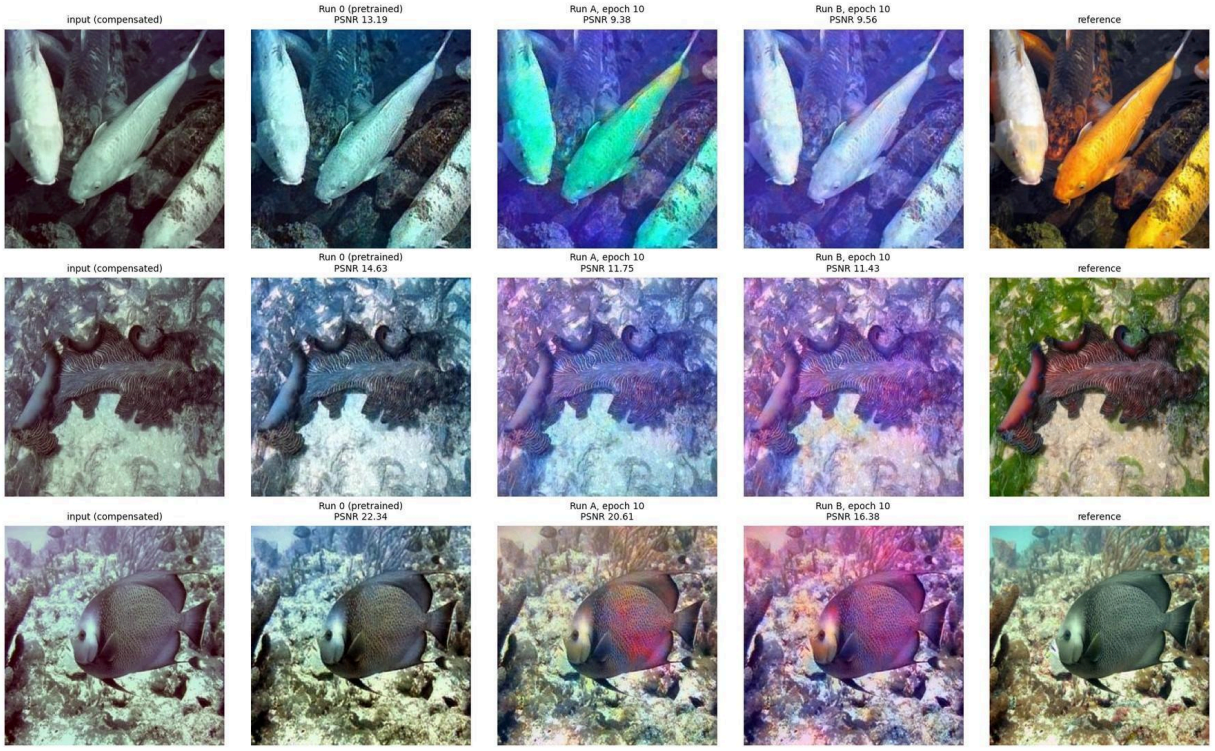


Figure 8: input, Run 0, Run A, Run B and reference for test images 1 to 3 (Run A and Run B at epoch 10). PSNR of each output is printed above it.

7. Analysis

Which fine-tuning run produced better outputs. Run A did, on PSNR. The mean PSNR over the 30 test images is 14.36 dB for Run A and 12.85 dB for Run B. The paired difference is 1.51 dB with a standard error of 0.40, and Run A is higher on 25 of the 30 images. SSIM does not separate the two runs (0.770 against 0.763, difference 0.007 with a standard error of 0.008). So replacing $\bar{\alpha}_t$ by $\bar{\alpha}_t^2$ made the outputs of the fine-tuned model measurably less close to the references, and we cannot say that it changed structural similarity.

At which checkpoint the difference first became visible. The earliest recorded difference appears at epoch 2, but its direction is not consistent across later checkpoints. On the three sample images, Run B's PSNR is already 1.3 dB below Run A's at epoch 2 (12.81 against 14.10). At epoch 6, however, Run B is higher, at epoch 8 the two are equal, and at epoch 10 Run B is 1.45 dB lower again (Table 4). In both runs the colours of the same image change from one checkpoint to the next (cyan, purple and green casts in Figures 4a and 4b), and the PSNR of a single image moves by up to about 5 dB between checkpoints of the same run (for example test image 2 in Run B, from 11.1 to 16.7 dB). That is larger than the gap between A and B. Three images are too few to separate a schedule effect from this checkpoint-to-checkpoint colour swing, so the 30-image evaluation at epoch 10 is the reliable comparison. What is visible from the very start is a property of the schedule and not of training: at the same t the noisy sample of Run B is noisier and shows structure later. At t = 500 the image structure is clearly visible in Run A and still faint in Run B (Figure 5).

Mechanistic explanation from the schedule mathematics. The explanations below fit the schedule mathematics and what we saw, but we did not test them one by one.

1. *The pretrained time conditioning no longer matches.* The pretrained network learned that timestep index t means an SNR of $\mathrm{SNR}_A(t)$. In Run B the same index means an SNR that is lower by the

factor $\bar{\alpha}_t/(1+\bar{\alpha}_t)$: 3 dB at t = 1, 11.2 dB at t = 1000 and 43.6 dB at t = T (Section 3). The network therefore receives noise levels that it connects with a different t, and fine-tuning has to learn this again. 2000 steps at a learning rate of 1e-5 can do that only partly. The post-hoc validation fits this picture, without proving it: with the pretrained weights, the loss under θ_t^2 is higher than under θ_t at every timestep we scored (for example 0.173 against 0.102 at t = 300), and fine-tuning lowers it by about 50 % in Run B, against 5.5 % in Run A.

2. *Fewer steps in the informative range.* In Run B, 52 % of the steps are almost pure noise ($\bar{\alpha} < 0.01$), against 32 % in Run A, and the SNR falls below 0 dB at t = 373 instead of t = 527. Global colour and layout are usually decided early in the reverse process, where little signal is left, so Run B has fewer steps in which the image can take shape.
3. *Error amplification.* At each reverse step the model estimates the clean image from its noise prediction $\hat{\varepsilon}$ (Eq. (14) of the paper):

$$\hat{\mathbf{x}}_0 = \frac{1}{\sqrt{\gamma_t}}\left(\mathbf{x}_t - \sqrt{1-\gamma_t}\, f_\theta(\mathbf{y}, \mathbf{x}_t, t)\right) \tag{14}$$

An error in the predicted noise is multiplied by $\sqrt{((1-\gamma_t)/\gamma_t)}$, which approaches $1/\sqrt{\gamma_t}$ at large t. This factor reaches 22,800 in Run B against 151 in Run A. The clipping of $\hat{x}_0$ to [−1, 1] hides some of the effect but does not remove it. This is consistent with the global colour casts that dominate the errors in both runs, but on the three sample images we did not find clearly more erratic colour in Run B than in Run A.

A cheap way to test explanation 1 would be to remap the timestep that is fed into the network in Run B, so that the pretrained network sees the SNR it expects. We did not run this.

How Run 0 compares with Run A and Run B, and what it says about fine-tuning on this dataset. Run 0 scores highest (PSNR 16.91 dB), then Run A (14.36 dB), then Run B (12.85 dB). Ten epochs of fine-tuning on 800 EUVP pairs did not improve the generated images of the held-out Underwater Dark test set under these metrics: PSNR dropped by 2.55 dB (standard error 0.53), and SSIM by 0.022, which is not distinguishable from zero. The size of this fine-tuning effect is larger than the effect of the schedule (1.51 dB). Several things are worth saying about this.

- **The training loss did not go down.** It rose from 0.0563 to 0.0636 in Run A and from 0.0447 to 0.0501 in Run B (about 12 to 13 %; one epoch mean has a noise of roughly 0.003, so the rise is modest, not negligible). Run A and Run B use the same seeds, so their curves move together (Figure 3) and do not confirm each other independently. The post-hoc validation shows that the objective itself did improve on unseen pairs (Run A about −5.5 %, Run B about −50.1 %), so the rising training loss is not a sign that the model got worse at the task it was trained for. The timesteps drawn in an epoch change the epoch mean a lot, because the loss at t = 100 is about 16 times the loss at t = 1900.
- **The noise-prediction loss and the image quality disagree.** The objective improved on held-out pairs, but the generated images moved further from the references. So the drop in PSNR is not explained by a failure to fit the objective. For Run A the validation shows where the objective got worse: at the highest noise levels (t ≥ 1500), which is where global colour and layout are decided. This fits the colour casts in Figures 4a and 4b, but we did not test it. Other possible reasons are that the average loss is dominated by small t and says little about a 2000-step sample, a learning rate (1e-5) above the 3e-6 of the authors, a fresh Adam state without warm-up, a batch of only 4, and EUVP targets whose colour style differs from the pretraining data. We ran no sweep, so we cannot say which one it is.
- **The “out-of-distribution” premise is supported, with one caveat.** DiffWater was trained on LSUI and UIEB, and EUVP is not mentioned in the paper (Section 2), so we treat EUVP as outside its pretraining data. The caveat is about file names: 13 of our 30 test image names and 2,399 of the 7,700 pool names follow an ImageNet-style pattern (..._nXXXXXXXX_...). If these images come from ImageNet or Flickr, some overlap with web-collected pretraining images cannot be fully excluded. We did not check where they come from, so we call EUVP very probably, but not certainly, outside the pretraining distribution.
- **Even the unprocessed input beats every model.** The colour-compensated input scores 20.65 dB against 16.91 dB for the best model (Run 0). PSNR and SSIM reward staying close to the reference, and a model that visibly changes colours is punished heavily. In Figures 4a and 4b, for example, the

centre fish of test image 1 is white in the input and orange in the reference, but comes out cyan in Run A and pale blue in Run B at epoch 10. A likely reason why the input is such a strong baseline is how EUVP is built: the degraded image of each pair was generated from the reference by a CycleGAN-based model (EUVP paper, Section 3.3), so it stays close to the reference by construction. We did not test this. In any case, these metrics measure how close an output is to EUVP's reference more than how good it looks.

Limits of this study. One seed, one exponent (x = 2.00), one fine-tuning budget (2000 steps), one learning rate, 30 test images from one subset, no validation set during fine-tuning (the post-hoc one is a single set of 200 pairs and its Run A and Run B values are not comparable with each other), and a model that is not the one we declared. The comparison of A and B holds for this setting.

8. Takeaway

- **The assignment is a controlled comparison, not a search for a better enhancer.** We compared the two schedules under identical conditions (same checkpoint, data, data order, seeds, hyperparameters, sampler and test images), and used Run 0 and Run A as controls.
- **Result.** Raising the schedule parameter to the power 2 lowered PSNR by about 1.5 dB relative to the original schedule after the same short fine-tune, without a measurable change in SSIM. Fine-tuning itself, with our settings, did not improve the generated images of the test set, although it lowered the noise-prediction loss on held-out validation pairs.
- **A schedule change is not a drop-in change for pretrained weights.** The timestep input of the network is tied to the schedule it was trained with. Changing the schedule changes what each t means, and the network has to learn that again. The validation loss of Run B roughly halves during fine-tuning while that of Run A drops by about 5 %, which shows how much more has to be learned again under the modified schedule.
- **A control run is necessary.** Comparing Run B only with the pretrained model would have put the whole 4 dB drop on the schedule. Run A shows that more than half of it comes from the fine-tune itself.
- **The training objective and the sample quality can disagree.** The loss on unseen pairs fell in both runs while PSNR on the test images fell too. Judging a fine-tune by its loss alone would have called both runs a success.
- **Engineering lessons.** The schedule buffers live inside the checkpoint and must be set again after loading. Fingerprints of the data split and the script catch silent differences between sessions. Saving per-image results lets a long evaluation survive disconnects.

Pooling GPU resources

We also learned a lot about getting experiments done on a limited GPU budget. We started with the free T4 GPUs on Colab, but the jobs either stopped with “resource exhausted” errors or were disconnected for being idle. That pushed us to try a provisioned T4 on Colab Enterprise, but it cannot mount Google Drive (`drive.mount` raises `NotImplementedError`) and using it would have meant paying for Google Cloud storage, which we did not want. So the team members used several Google accounts and ran the GPU jobs in a distributed way. The script saves a resume file every two epochs (`resume.pt`) and every finished evaluation image, all on a shared Drive folder. When the quota of one account ran out, another account picked up the job from there and finished it. We also made the launch cell a blocking cell, because a background process was disconnected by Colab as idle.

9. Contribution Statement

This was group work from the first meeting to the last page, and we want it to be read as the work of all four of us together. We had several meetings to choose the assignment topic; to choose the pretrained model and the dataset from our individual search lists; to go through the DiffWater code and understand how it works; to decide the changes needed so that our dataset could be used by the model (the data loading, the schedule change, the logging and the resume function); to plan and check the runs; and to prepare this report.

The GPU work was shared. Team members ran jobs from their own Google accounts, and when a run was interrupted or an account ran out of quota, the saved resume file and the images written so far were picked up from the shared Drive folder by another account, which finished the job. Run A, for example,

was finished in two other accounts after the first one ran out of quota. So Run 0, Run A and Run B are the result of the combined effort of the group, and we do not attribute them to one person. All four of us contributed equally to all the work items and to the completion of the assignment.

References

1. M. Guan, H. Xu, G. Jiang, M. Yu, Y. Chen, T. Luo and X. Zhang, “DiffWater: Underwater Image Enhancement Based on Conditional Denoising Diffusion Probabilistic Model,” *IEEE J. Sel. Topics Appl. Earth Obs. Remote Sens.*, vol. 17, pp. 2319–2335, 2024, doi 10.1109/JSTARS.2023.3344453. Code: https://github.com/Guan-MS/DiffWater
2. M. J. Islam, Y. Xia and J. Sattar, “Fast Underwater Image Enhancement for Improved Visual Perception” (introduces FUnIE-GAN and the EUVP dataset), arXiv:1903.09766. Dataset page: https://irvlab.cs.umn.edu/resources/euvp-dataset
3. UW-DiffPhys, “Underwater Image Enhancement with Physical-based Denoising Diffusion Implicit Models,” arXiv:2409.18476 (the model named in our first declaration). Code: https://github.com/bachzz/UW-DiffPhys
4. J. Ho, A. Jain and P. Abbeel, “Denoising Diffusion Probabilistic Models,” NeurIPS 2020 (the DDPM that DiffWater builds on, reference [15] of the DiffWater paper).